

INF264 - Homework 5

Credits: Pierre Gillot, Natacha Galmiche, Odin Hoff Gardå

This homework consists of two main parts. In the first part, you will manually perform the learning process of a simple neural network, and see that doing so reduces the loss function of the network. In the second part, you will build and train an MLP (Multi-layer Perceptron) model using `sklearn` to solve a regression task.

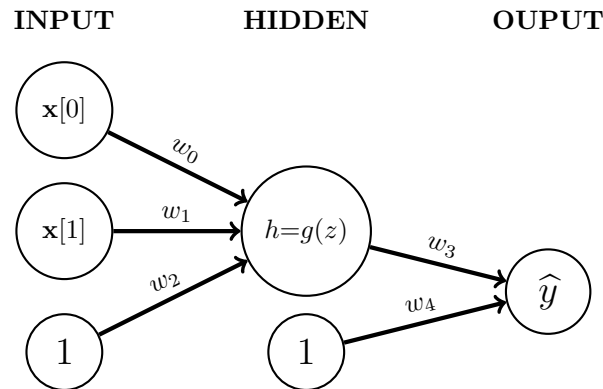
1 Gradient descent and backpropagation in a simple neural network

Usually learning a neural network on some data consists of two steps:

- i A **forward pass**, in which the data is processed in the neural network from the input layer to the output layer.
- ii A **backward pass**, in which the error is propagated from the output layer to the input layer in order to compute the loss function's gradient (backpropagation algorithm).

Finally, a gradient descent method is used on the network's parameters (weights and biases) in order to minimize the loss.

Consider the following neural network with one hidden layer:



where:

- $\mathbf{x} = (\mathbf{x}[0], \mathbf{x}[1]) \in \mathbb{R}^2$ is a feature vector from the data and represents the input layer.
- $\theta = (w_0, w_1, w_2, w_3, w_4)$ are the parameters of the network. Note that w_2, w_4 represent the biases¹, while w_0, w_1, w_3 represent regular weights.
- $g: \mathbb{R} \rightarrow \mathbb{R}_{\geq 0}$ is the ReLU (Rectified Linear Unit) activation function, defined by $g(t) = \max(0, t)$.
- z is the input of the hidden layer:

$$z(\mathbf{x}) = w_0 \cdot \mathbf{x}[0] + w_1 \cdot \mathbf{x}[1] + w_2 \cdot 1.$$

¹Adding ones to the input and treating the biases as weights like this is sometimes referred to as *the bias trick*.

- h is the output of the hidden layer:

$$h(\mathbf{x}) = g(z(\mathbf{x})).$$

- $\hat{y}$ is the output of the network, the prediction of $\mathbf{x}$:

$$\hat{y}(\mathbf{x}) = w_3 \cdot h(\mathbf{x}) + w_4 \cdot 1.$$

We define the loss function of this neural network as the mean-squared error. That is, for a dataset $\mathcal{D}$ containing the feature vectors $\mathbf{x}_1, \dots, \mathbf{x}_N$ and their respective targets $y_1, \dots, y_N$, the loss function $\mathcal{L}$ on this data is defined as:

$$\mathcal{L}(\mathcal{D}) = \frac{1}{N} \sum_{n=1}^N L(\mathbf{x}_n, y_n),$$

where

$$L(\mathbf{x}_n, y_n) = (\hat{y}(\mathbf{x}_n) - y_n)^2.$$

The gradient of the neural network's loss function with respect to the network's parameters θ on the dataset $\mathcal{D}$ is then:

$$\nabla_{\theta} \mathcal{L}(\mathcal{D}) = \frac{1}{N} \sum_{n=1}^N \nabla_{\theta} L(\mathbf{x}_n, y_n),$$

where

$$\nabla_{\theta} L(\mathbf{x}_n, y_n) = \left(\frac{\partial L(\mathbf{x}_n, y_n)}{\partial w_0}, \frac{\partial L(\mathbf{x}_n, y_n)}{\partial w_1}, \frac{\partial L(\mathbf{x}_n, y_n)}{\partial w_2}, \frac{\partial L(\mathbf{x}_n, y_n)}{\partial w_3}, \frac{\partial L(\mathbf{x}_n, y_n)}{\partial w_4} \right).$$

Once the gradient of the neural network's loss function with respect to the parameters θ is computed, a gradient descent method is applied to update the parameters in an attempt to minimize the loss:

$$\theta \leftarrow \theta - \eta \cdot \nabla_{\theta} \mathcal{L}(\mathcal{D}),$$

where $\eta > 0$ is the learning rate of the gradient descent.

The critical aspect when learning a neural network is thus to compute efficiently the loss gradient $\nabla_{\theta} L(\mathbf{x}, y)$ for each individual sample $(\mathbf{x}, y)$ in the training set $\mathcal{D}$, and is the purpose of the backpropagation algorithm, which works as follows:

For a given sample $(\mathbf{x}, y)$, define the error in the output layer and the error in the hidden layer as

$$\delta_{\hat{y}}(\mathbf{x}, y) = \frac{\partial L(\mathbf{x}, y)}{\partial \hat{y}}, \quad \delta_h(\mathbf{x}, y) = \frac{\partial L(\mathbf{x}, y)}{\partial z} \quad \text{respectively.}$$

We can show that the error in the hidden layer propagates from the error in the output layer. Indeed, L is a function of $\hat{y}$, $\hat{y}$ is a function of h and h is a function of z , so the chain rule yields:

$$\begin{aligned} \delta_h(\mathbf{x}, y) &= \frac{\partial L(\mathbf{x}, y)}{\partial z} \\ &= \frac{\partial L(\mathbf{x}, y)}{\partial \hat{y}} \cdot \frac{\partial \hat{y}(\mathbf{x})}{\partial h} \cdot \frac{\partial h(\mathbf{x})}{\partial z} \\ &= \delta_{\hat{y}}(\mathbf{x}, y) \cdot \frac{\partial (w_3 \cdot h(\mathbf{x}) + w_4 \cdot 1)}{\partial h} \cdot \frac{\partial g(z(\mathbf{x}))}{\partial z} \\ &= \delta_{\hat{y}}(\mathbf{x}, y) \cdot w_3 \cdot g'(z(\mathbf{x})), \end{aligned}$$

where g' is the left derivative of the ReLU activation g , defined as

$$t \mapsto \begin{cases} 1 & \text{if } t > 0 \\ 0 & \text{if } t \leq 0. \end{cases}$$

Applying the chain rule also gives the partial derivatives of L with respect to the network's parameters:

$$\frac{\partial L(\mathbf{x}, y)}{\partial w_0} = \delta_h(\mathbf{x}, y) \cdot \mathbf{x}[0] \quad (1)$$

$$\frac{\partial L(\mathbf{x}, y)}{\partial w_1} = \delta_h(\mathbf{x}, y) \cdot \mathbf{x}[1] \quad (2)$$

$$\frac{\partial L(\mathbf{x}, y)}{\partial w_2} = \delta_h(\mathbf{x}, y) \quad (3)$$

$$\frac{\partial L(\mathbf{x}, y)}{\partial w_3} = \delta_{\hat{y}}(\mathbf{x}, y) \cdot h(\mathbf{x}) \quad (4)$$

$$\frac{\partial L(\mathbf{x}, y)}{\partial w_4} = \delta_{\hat{y}}(\mathbf{x}, y) \quad (5)$$

In all that follows, we consider the dataset $\mathcal{D}$ containing two samples:

i $\mathbf{x}_1 = (2, 1) \quad y_1 = 1.3$

ii $\mathbf{x}_2 = (-3, 2) \quad y_2 = 1.9$

We also assume that the parameters of the network are initialized as

$$\theta = (w_0, w_1, w_2, w_3, w_4) = (1, 1, 0, 1, 0).$$

Once you have carefully read the explanations above, answer the following questions:

1. Perform a forward pass on the dataset $\mathcal{D}$: compute the predictions $\hat{y}(\mathbf{x}_1)$ and $\hat{y}(\mathbf{x}_2)$.
2. Compute the loss of the network on the dataset $\mathcal{D}$, that is $\mathcal{L}(\mathcal{D})$.
3. Prove the backpropagation equalities (1), (2), (3), (4), (5) using the chain rule.

Hint 1: For the first three equalities, justify that you can use the chain rule

$$\frac{\partial L}{\partial w_i} = \frac{\partial L}{\partial z} \cdot \frac{\partial z}{\partial w_i}, \quad i \in \{0, 1, 2\}.$$

Hint 2: For the two last equalities, justify that you can use the chain rule

$$\frac{\partial L}{\partial w_i} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial w_i}, \quad i \in \{3, 4\}.$$

Hint 3: If you struggle, ask for our help and do not get stuck for hours on this!

4. Compute the gradient of the neural network's loss function, that is $\nabla_{\theta} \mathcal{L}(\mathcal{D})$.
Hint 4: Compute first the gradients $\nabla_{\theta} L(\mathbf{x}_1, y_1)$ and $\nabla_{\theta} L(\mathbf{x}_2, y_2)$ using the backpropagation rules, then compute their mean.
Hint 5: During backpropagation, you should first compute the error in the output layer, then propagate this error to obtain the error in the hidden layer, and finally use the equalities (1), (2), (3), (4), (5) to get the gradients.
5. Perform the gradient descent update on the parameters $\theta = (w_0, w_1, w_2, w_3, w_4)$, with a learning rate $\eta = 0.01$.
6. Recompute the loss of the network on the dataset $\mathcal{D}$ and check that it decreased after the network's parameters were updated.
7. Intuitively, what is the effect of the learning rate hyper-parameter η on the update of the parameters θ ?

2 Neural networks for a regression task using scikit-learn

In this second section, we will do implementation using `sklearn`. More specifically, we will first visualize and pre-process the diabetes dataset. Then, we will build different MLPs (Multi-Layer Perceptron) regressors that consist of a few dense layers, each model defined with a different set of hyper-parameters. Then, we will select the best model based on their performance on the diabetes dataset. Finally, we will evaluate its performance on unseen data and visualize its predictions compare to the expected values.

Note: There is no template for this exercise, but you are free to get inspiration from previous homework solutions.

1. Load the diabetes dataset using the `sklearn.datasets.load_diabetes` function. Alternatively, you can find the dataset following this link: <https://www4.stat.ncsu.edu/~boos/var.select/diabetes.html>.
2. Visualize the dataset. You can use scatterplots for example.
3. Preprocess the data:
 - (a) Select relevant features.
 - (b) Split the data into a training, validation and test dataset.
 - (c) Scale the data (you can use `MinMaxScaler` and/or `StandardScaler` from `sklearn.preprocessing`).
4. Build simple MLPs using `sklearn`:
 - (a) Use the `MLPRegressor` class from `sklearn.neural_network`
 - (b) Define different sets of hyperparameters. You can for example play with the `hidden_layer_sizes`, `momentum`, `solver` and `learning_rate_init` hyperparameters.
5. Train all your models using a loss function suitable for a regression task
6. Select the best model using an appropriate metric and K-fold cross-validation.
7. Evaluate the best model using the same metric. Can you really tell how good your model is performing based on the metric here?
8. Visualizing the performance of your model.:
 - (a) Plot the model predictions next to the expected values, both for the train-validation dataset and the test dataset. You can use different colors to distinguish between the expected values and the predicted ones. Comment. Can you see if your model is doing great? Can you see which cases are hard to predict?
 - (b) Plot the features/targets in a similar fashion than in question 2. More specifically, define multiple scatterplots, where the y-axis is always the target value and each x-axis is one of the selected features. The data is taken from the test dataset and the N best predictions are colored in green, the N worst in red and all the others in blue. Comment. Can you see which cases are hard to predict now?